

DSPO: STABLE AND EFFICIENT POLICY OPTIMIZATION FOR AGENTIC SEARCH AND REASONING

Anonymous authors
Anonymous submission

ABSTRACT

Equipping LLMs with active external search is essential for complex, real-world tasks that exceed static parametric knowledge. Existing approaches either depend on prompting to elicit latent agent behavior or expose clear performance ceilings and collapse modes when RL is applied to interactive search, leaving substantial agentic capability unrealized. We introduce **Dynamic-filter Sequence-level Policy Optimization (DSPO)**, a robust RL algorithm that directly stabilizes agent training through sequence-level optimization and dynamic sample filtering. DSPO trains the model purely from RL feedback to interleave multi-turn search and reasoning, eliminating the need for supervised demonstration data. Across multiple QA benchmarks, our 7B model delivers a **34.1%** relative improvement over comparable prior work and even surpasses the 14B model from prior work on complex multihop QA such as HotpotQA by nearly **9% relative**, while maintaining exceptional training stability.

1 INTRODUCTION

Large Language Models (LLMs) (Brown et al., 2020; Touvron et al., 2023; Zhao et al., 2023) now achieve strong performance across specialized domains such as math (Shao et al., 2024; Trinh et al., 2024; Yu et al., 2025), coding (Zheng et al., 2023; Yang et al., 2024), and creative writing (Chakrabarty et al., 2024; Marco et al., 2024). Yet their central limitation remains decisive: their knowledge is static and bounded by training data. A direct way to break this ceiling is to equip LLMs with search, turning them into agents that actively query external knowledge sources (Jin et al., 2025b). This capability is a canonical form of tool-calling (Schick et al., 2023), in which the model must decide when and how to interact with an external search tool to solve questions it cannot answer from memory alone. Learning this behavior requires a complex multi-step policy, making agentic search a natural and demanding setting for Reinforcement Learning (RL).

Supervised Fine-Tuning (SFT) depends on costly static demonstrations and does not reliably teach exploration, whereas RL gives LLMs a mechanism to discover effective policies through trial-and-error (Chu et al., 2025). This has made value-free methods such as Group Relative Policy Optimization (GRPO) (Shao et al., 2024) attractive because they reduce memory overhead and simplify training. However, the open-ended search setting exposes limitations that are muted in more constrained tasks (Jin et al., 2025b; Yu et al., 2025; Cui et al., 2025; Liu et al., 2025). The failure mode has two concrete sources. First, as identified by Zheng et al. (2025), GRPO optimizes token-level ratios against sequence-level rewards, producing high-variance gradients that can destabilize training. Second, sparse search rewards frequently produce homogeneous rollout groups, such as all successes or all failures, which collapse the advantage signal and waste samples (Yu et al., 2025; Liu et al., 2025).

We address these failure modes with **Dynamic-filter Sequence-level Policy Optimization (DSPO)**. DSPO turns two recent policy-optimization principles into a unified algorithm for agentic search. It adopts sequence-level optimization from GSPO (Zheng et al., 2025) so that the optimization unit matches the reward unit, directly stabilizing long-horizon search-and-reasoning updates. It also incorporates a dynamic outcome-based filtering mechanism inspired by DAPO (Yu et al., 2025), actively constructing training batches from rollout groups that contain both successful and unsuccessful outcomes. This design gives $\hat{A}_i$ a consistently informative signal instead of allowing sparse rewards to erase the update. Together, these components make DSPO a stable, high-performance algorithm for complex multi-turn search and reasoning. Empirically, our model achieves a **34.1% relative improvement** over a leading 7B baseline (Jin et al., 2025b) and surpasses its 14B counterpart (Jin et al., 2025a) on HotpotQA by **nearly 9% relative** (0.613 vs. 0.563).

In summary, our main contributions are as follows:

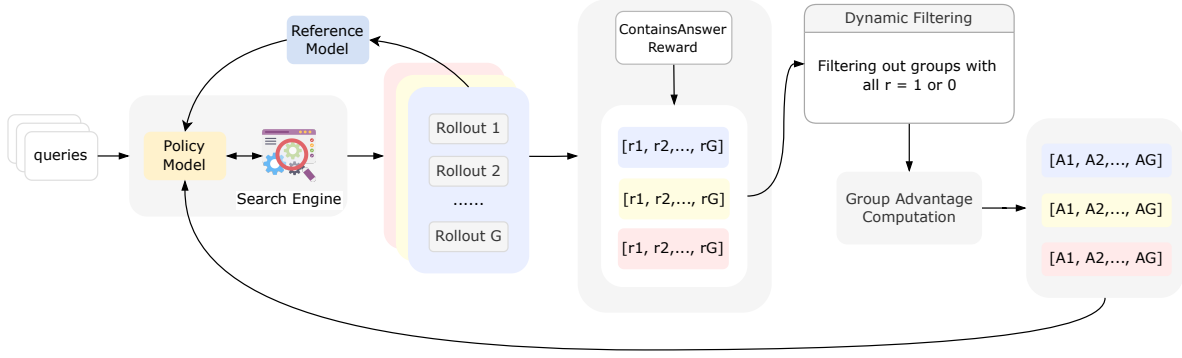


Figure 1: Overview of the DSPO training loop. For each query, the policy model generates a group of G trajectories through interaction with the search environment, and each trajectory receives a sparse terminal reward. The **dynamic filter** removes homogeneous-outcome groups and continues sampling until the batch is filled, ensuring that every training batch carries an effective advantage signal. The resulting advantages update the policy through the sequence-level objective.

- We introduce DSPO, a strengthened RL algorithm that directly resolves the central instability and sample-inefficiency obstacles in training agentic search models. DSPO unifies two decisive principles in one cohesive framework: **sequence-level optimization** for robust policy updates and **dynamic outcome-based filtering** for a dense and effective learning signal.
- We establish DSPO’s substantial performance gains through rigorous benchmarking. Our 7B model achieves a **34.1% relative improvement** over a comparable 7B baseline and, more strikingly, outperforms its 14B counterpart on complex multi-hop QA, achieving a nearly **9% relative gain** on HotpotQA (0.613 vs. 0.563).
- We provide extensive empirical evidence that DSPO delivers superior training stability and a stable learning trajectory. Crucially, these gains are achieved with only a basic BM25 retriever, isolating the improvements to the robustness of the algorithm rather than retriever strength.

2 RELATED WORK

2.1 RL FOR LLMs

RL for LLMs has advanced rapidly, moving from foundational Reinforcement Learning from Human Feedback (RLHF) pipelines that use PPO and explicit reward models (Ouyang et al., 2022; Christiano et al., 2017; Schulman et al., 2017) toward simpler direct-optimization frameworks such as DPO (Rafailov et al., 2023). Group Relative Policy Optimization (GRPO) (Shao et al., 2024) marks a major shift toward value-free optimization by deriving the reward signal from group statistics rather than a value model. Nevertheless, GRPO’s token-level objective is now known to create training instability (Liu et al., 2025; Cui et al., 2025), motivating targeted remedies. GSPO shifts optimization to the sequence level to match the reward unit (Zheng et al., 2025), DAPO improves sparse-reward learning with dynamic outcome-based sampling (Yu et al., 2025), and GMPO uses geometric-mean aggregation to reduce token-level sensitivity to outliers (Zhao et al., 2025). In our setting, these advances still leave collapse modes or performance bottlenecks. DSPO therefore makes the stronger synthesis required for autonomous search agents: it combines sequence-level optimization with dynamic filtering and filling in a single algorithm built for stable, efficient search-policy learning.

2.2 LLMs WITH AGENTIC RETRIEVAL

Retrieval-augmented generation (RAG) directly mitigates the static-knowledge limitation of LLMs by integrating external retrievers that provide evolving information (Lewis et al., 2020; Gao et al., 2023). Classic RAG systems use dense retrievers to fetch relevant documents and concatenate them into the LLM input (Karpukhin et al., 2020), but these fixed pipelines restrict autonomy in complex multi-turn settings. The field has therefore moved toward agentic paradigms in which LLMs plan, search, and reason iteratively. ReAct couples reasoning with acting so LLMs can use tools for tasks such as web navigation (Yao et al., 2023), and multi-agent systems such as AutoGen enable collaborative workflows (Wu et al., 2024). Recent work on agentic RAG and RL integration further lets agents improve retrieval

through learned decisions: [Wu et al. \(2025\)](#) introduce Agentic Reasoning, and RL-based approaches ([Jin et al., 2025b](#); [Chen et al., 2025](#); [Song et al., 2025](#)) train LLMs to interleave reasoning and search using purely RL. This end-to-end paradigm internalizes agent behavior and avoids much of the engineering overhead of multi-agent pipelines. The remaining barrier is algorithmic: current RL methods still face instability and performance ceilings in open-ended search. DSPO targets this barrier directly with a robust and efficient RL algorithm for search-and-reasoning agents.

3 METHODOLOGY

This section formalizes agentic search as an RL problem and pinpoints why prior policy-optimization objectives fail in this setting. We then present **Dynamic-filter Sequence-level Policy Optimization (DSPO)**, detailing the two mechanisms that deliver stable and efficient training. Finally, we give the integrated training algorithm.

3.1 PRELIMINARIES

Policy Gradient Methods for LLMs. RL training for LLMs commonly uses policy-gradient methods such as PPO ([Schulman et al., 2017](#)), a widely used algorithm for LLM alignment. PPO updates a policy π_θ by maximizing a clipped surrogate objective using samples from an old policy $\pi_{\theta_{\text{old}}}$. Averaged over tokens, the objective is:

$$J_{\text{PPO}}(\theta) = \mathbb{E}_{x \sim \mathcal{D}, y \sim \pi_{\theta_{\text{old}}}(\cdot|x)} \left[\frac{1}{|y|} \sum_{t=1}^{|y|} \min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right], \quad (1)$$

where $r_t(\theta) = \frac{\pi_\theta(y_t|x, y_{<t})}{\pi_{\theta_{\text{old}}}(y_t|x, y_{<t})}$ is the token-level importance ratio. PPO’s reliance on a separately trained value model to estimate token-level advantages $\hat{A}_t$ via Generalized Advantage Estimation (GAE) ([Schulman et al., 2015](#)) adds substantial memory overhead and introduces another instability source.

GRPO ([Shao et al., 2024](#)) removes this value-model dependency by sampling a group of G responses $\{y_i\}_{i=1}^G$ for each prompt x and normalizing each response reward against group statistics. This design is more memory efficient, but it still optimizes at the token level:

$$J_{\text{GRPO}}(\theta) = \mathbb{E}_{x \sim \mathcal{D}, \{y_i\}_{i=1}^G \sim \pi_{\theta_{\text{old}}}(\cdot|x)} \left[\frac{1}{G} \sum_{i=1}^G \frac{1}{|y_i|} \sum_{t=1}^{|y_i|} \min \left(r_{i,t}(\theta) \hat{A}_i, \text{clip}(r_{i,t}(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_i \right) - \beta D_{KL}(\pi_\theta || \pi_{\text{ref}}) \right], \quad (2)$$

where $r_{i,t}(\theta) = \frac{\pi_\theta(y_{i,t}|x, y_{i,<t})}{\pi_{\theta_{\text{old}}}(y_{i,t}|x, y_{i,<t})}$, and the advantage for every token $y_{i,t}$ in a response y_i is set to the same sequence-level value:

$$\hat{A}_{i,t} = \hat{A}_i = \frac{R_i - \text{mean}(R)}{\text{std}(R)}, \quad (3)$$

The key mismatch is explicit: every token in response y_i receives the same advantage $\hat{A}_i$, although that advantage is derived from a sequence-level reward.

Agentic Search as a Markov Decision Process. We formulate agentic search and reasoning as a sequential decision-making problem, specifically a discrete-time, finite-horizon Markov Decision Process (MDP).

- **State (s_t):** At turn t , the state s_t contains the complete interaction history: the initial question q , all prior thoughts and search queries, and the evidence returned by the environment. Because the agent must interpret retrieved evidence and integrate it into later decisions, the state grows throughout the trajectory and creates long-horizon dependencies that conventional token-level RL handles poorly.
- **Action (a_t):** An action is a complete textual segment generated by π_θ : free-form reasoning followed by a decision. The segment ends either with a `</tool_call>` token, which triggers search and returns results within `</tool_response>`, or with a `</answer>` token, which terminates the trajectory. This structured action space requires the agent to learn not only *what* to generate but also *when* to search, producing substantial variation in trajectory length.

- **Policy (π_θ):** The policy π_θ is the underlying LLM, which generates tokens autoregressively conditioned on the state. Although the policy conditions on the full history, the optimization target is computed only over model-generated thoughts and actions, with retrieved environmental content masked out (Jin et al., 2025b). This policy demands credit assignment over long sequences where intermediate reasoning steps receive no direct supervision, making stable sequence-level optimization central rather than optional.
- **Trajectory (τ):** A trajectory $\tau = (s_1, a_1, \dots, s_T, a_T)$ records all reasoning and tool interactions for a question, including model-generated actions and environment-returned search results. Since the final reward is assigned to the whole trajectory, the optimization problem is intrinsically sequence-level: early search and reasoning decisions can determine final answer correctness.
- **Reward ($R(\tau)$):** We use a sparse terminal reward: a trajectory receives $R = 1$ when the final answer contains the ground-truth text and $R = 0$ otherwise:

$$R(\tau) = \begin{cases} 1 & \text{if } a_{\text{gold}} \subseteq a_{\text{pred}}, \\ 0 & \text{otherwise.} \end{cases} \quad (4)$$

RL with a Search Engine. Following Jin et al. (2025b), we model the search engine $\mathcal{S}$ as an explicit part of the environment. The policy LLM π_θ learns trajectories by interleaving reasoning with calls to $\mathcal{S}$. The optimization problem is to find a policy that maximizes expected reward while a KL divergence term prevents large deviations from the reference policy π_{ref} :

$$\max_{\pi_\theta} \mathbb{E}_{x \sim \mathcal{D}, y \sim \pi_\theta(\cdot|x; \mathcal{S})} [R(x, y)] - \beta D_{\text{KL}} [\pi_\theta(y|x; \mathcal{S}) || \pi_{\text{ref}}(y|x; \mathcal{S})]. \quad (5)$$

Here, $y \sim \pi_\theta(\cdot|x; \mathcal{S})$ denotes a trajectory generated by a multi-step process that combines policy token generation with information returned by the search engine $\mathcal{S}$.

Motivation for DSPO. Frameworks such as Search-R1 (Jin et al., 2025b) establish agentic search as an RL problem, but conventional PPO or GRPO-style optimization is not sufficient for this environment. Open-ended search amplifies token-level instability because the reward is assigned at the sequence level while the update is applied at the token level (Zheng et al., 2025). This mismatch produces high-variance gradients over long trajectories and can culminate in policy collapse. Sparse binary rewards create a second bottleneck: many batches contain only successful or only unsuccessful trajectories, yielding abnormal advantages and no useful learning signal (Yu et al., 2025; Liu et al., 2025). DSPO is designed precisely to neutralize these two failure modes.

3.2 DYNAMIC-FILTER SEQUENCE-LEVEL POLICY OPTIMIZATION

DSPO advances prior methods with two tightly coupled innovations: (1) it optimizes policies at the sequence level, aligning the training objective with trajectory-level rewards, and (2) it dynamically filters rollout groups so each training batch provides a high-quality, non-zero learning signal. Figure 1 shows how these components operate within the full training loop.

3.2.1 SEQUENCE-LEVEL POLICY OPTIMIZATION FOR ENHANCED STABILITY

Inspired by GSPO (Zheng et al., 2025), DSPO replaces the unstable token-level importance ratio with a sequence-level counterpart that matches the reward granularity. For response y_i , the sequence-level importance ratio $s_i(\theta)$ is the geometric mean of its token-level ratios:

$$s_i(\theta) = \left(\frac{\pi_\theta(y_i|x)}{\pi_{\theta_{\text{old}}}(y_i|x)} \right)^{\frac{1}{|y_i|}} = \exp \left(\frac{1}{|y_i|} \sum_{t=1}^{|y_i|} \log \frac{\pi_\theta(y_{i,t}|x, y_{i,<t})}{\pi_{\theta_{\text{old}}}(y_{i,t}|x, y_{i,<t})} \right). \quad (6)$$

This length normalization reduces variance and keeps $s_i(\theta)$ in a consistent numerical range across sequence lengths, which makes clipping substantially more stable.

Gradient Analysis. The gradient comparison makes the stability advantage of DSPO explicit. The gradient of the token-level GRPO objective (unclipped) scales each token’s log-probability gradient by a noisy, token-specific weight $r_{i,t}(\theta)$. By contrast, the DSPO sequence-level objective scales the average log-probability gradient of the full sequence

by one more stable sequence-level weight $s_i(\theta)$:

$$\nabla_{\theta} J_{\text{GRPO}} \propto \mathbb{E} \left[\hat{A}_i \cdot \sum_{t=1}^{|y_i|} r_{i,t}(\theta) \nabla_{\theta} \log \pi_{\theta}(y_{i,t} | \dots) \right] \quad (7)$$

$$\nabla_{\theta} J_{\text{DSPO}} \propto \mathbb{E} \left[\hat{A}_i \cdot s_i(\theta) \cdot \sum_{t=1}^{|y_i|} \nabla_{\theta} \log \pi_{\theta}(y_{i,t} | \dots) \right] \quad (8)$$

By applying one holistic correction factor to the entire trajectory, DSPO avoids the accumulation of token-level noise that destabilizes prior methods and yields fundamentally more stable training. In parallel, the dynamic filtering mechanism guarantees an informative advantage signal $\hat{A}_i$ by constructing training batches from rollout groups that contain both successes and failures, preventing wasted samples in sparse-reward environments.

3.2.2 DYNAMIC OUTCOME-BASED FILTERING FOR EFFICIENT LEARNING

The sparse binary reward creates a direct challenge for group-based advantage estimation. If all G responses in a group are correct ($R = 1$) or all are incorrect ($R = 0$), the normalized advantage $\hat{A}_i$ becomes zero or undefined. These batches provide no useful gradient signal and therefore waste computation.

DSPO eliminates this inefficiency with a dynamic filtering mechanism inspired by DAPO (Yu et al., 2025). During sampling, DSPO retains only trajectory groups that contain both successful and unsuccessful outcomes. A group $\{y_i\}_{i=1}^G$ enters training only when its rewards $\{R_i\}_{i=1}^G$ satisfy:

$$0 < \sum_{i=1}^G R_i < G. \quad (9)$$

This condition ensures non-zero reward variance in every training group and guarantees a meaningful advantage signal. The resulting dynamic selection curates a high-quality dataset for each policy update, turning sparse terminal feedback into an efficient learning signal.

3.3 THE DSPO OBJECTIVE AND TRAINING ALGORITHM

Combining sequence-level optimization with dynamic filtering yields the final DSPO objective. For each valid group from the filtered sample space $\mathcal{D}_{\text{filtered}}$, DSPO computes $\hat{A}_i$ through group-relative normalization:

$$\hat{A}_i = \frac{R_i - \text{mean}(R)}{\text{std}(R) + \delta}, \quad (10)$$

where δ is a small constant for numerical stability. DSPO updates the policy π_{θ} by maximizing:

$$J_{\text{DSPO}}(\theta) = \mathbb{E}_{(x, \{y_i\}) \in \mathcal{D}_{\text{filtered}}} \left[\frac{1}{G} \sum_{i=1}^G \min \left(s_i(\theta) \hat{A}_i, \text{clip}(s_i(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_i \right) \right], \quad (11)$$

where s_i is the sequence-level importance ratio defined as the geometric mean of the token-level ratios. During likelihood calculation, we follow Jin et al. (2025b) and mask all tokens retrieved from the search tool. This forces the model to learn how to use external knowledge for reasoning rather than merely reproduce retrieved text. Algorithm 1 details the full training process.

4 EXPERIMENTS

This section empirically establishes the effectiveness of Dynamic-filter Sequence-level Policy Optimization (DSPO). The experiments are designed to show that DSPO: (1) achieves strong performance on challenging question-answering benchmarks; (2) substantially improves training stability and avoids the collapse observed in baseline methods; and (3) derives its gains from the combined effect of sequence-level optimization and dynamic filtering.

4.1 EXPERIMENTAL SETUP

Following Search-R1 (Jin et al., 2025b), we use the prompt template in Table 1 to define the model’s search-task actions, including `<think>`, `<tool_call>`, and `<answer>`.

Algorithm 1 Dynamic-filter Sequence-level Policy Optimization (DSPO)

```

1: Input: Initial policy  $\pi_{\theta_0}$ , fixed reference policy  $\pi_{\text{ref}}$ , prompt dataset  $\mathcal{D}$ , group size  $G$ , batch size  $B$ , search tool  $\mathcal{R}$ .
2: Initialize policy  $\pi_{\theta} \leftarrow \pi_{\theta_0}$ .
3: for each training step do
4:    $\pi_{\theta_{\text{old}}} \leftarrow \pi_{\theta}$ .
5:   Initialize training buffer  $\mathcal{B} \leftarrow \emptyset$ .
6:   while  $|\mathcal{B}| < B$  do
7:     Sample a prompt  $x \sim \mathcal{D}$ .
8:     Generate a group of  $G$  trajectories  $\{y_i\}_{i=1}^G$  using  $\pi_{\theta_{\text{old}}}$  and the search tool  $\mathcal{R}$ .
9:     Compute terminal rewards  $\{R_i\}_{i=1}^G = \{\text{ContainsAnswer}(y_i, y_{\text{gold}})\}_{i=1}^G$ .
10:    if  $0 < \sum_{i=1}^G R_i < G$  then ▷ Dynamic outcome-based filtering
11:      Add  $(x, \{y_i\}_{i=1}^G, \{R_i\}_{i=1}^G)$  to  $\mathcal{B}$ .
12:    end if
13:  end while
14:  for each  $(x, \{y_i\}, \{R_i\})$  in  $\mathcal{B}$  do
15:    Compute advantages  $\{\hat{A}_i\}_{i=1}^G$  via group normalization of  $\{R_i\}$ .
16:    Compute sequence-level importance ratios  $\{s_i(\theta)\}_{i=1}^G$  using Eq. 6.
17:    Compute the DSPO loss for the group using Eq. 11, applying masks to retrieved tokens.
18:  end for
19:  Update policy parameters  $\theta$  by taking a gradient step on the total loss from  $\mathcal{B}$ .
20: end for

```



Figure 2: **Validation performance of DSPO across seven benchmarks during training.** Accuracy increases steadily during training, showing that DSPO’s reward gains translate into stronger generalization and a robust search-and-reasoning policy.

Benchmarks and Baselines. For a rigorous and directly comparable evaluation, we follow the established Search-R1 protocol (Jin et al., 2025b). We train on a composite dataset containing the training splits of Natural Questions (NQ) (Kwiatkowski et al., 2019) and HotpotQA (Yang et al., 2018). We then evaluate generalization on seven diverse QA test sets: NQ, TriviaQA (Joshi et al., 2017), PopQA (Mallen et al., 2022), HotpotQA, 2WikiMultiHopQA (Ho et al., 2020), Musique (Trivedi et al., 2022), and Bamboogle (Press et al., 2022).

Our comparison suite includes strong external baselines and diagnostic internal ablations. The external baselines are Qwen2.5-7B and 14B models trained with PPO and GRPO in the Search-R1 framework (Jin et al., 2025b;a). To isolate DSPO’s design choices, we include two internal ablations: (1) **DSPO w/o dynamic filter**, which is equivalent to GSPO (Zheng et al., 2025), and (2) **DSPO w/o sequence-level opt.**, which reverts to the strong token-level policy used by DAPO (Yu et al., 2025).

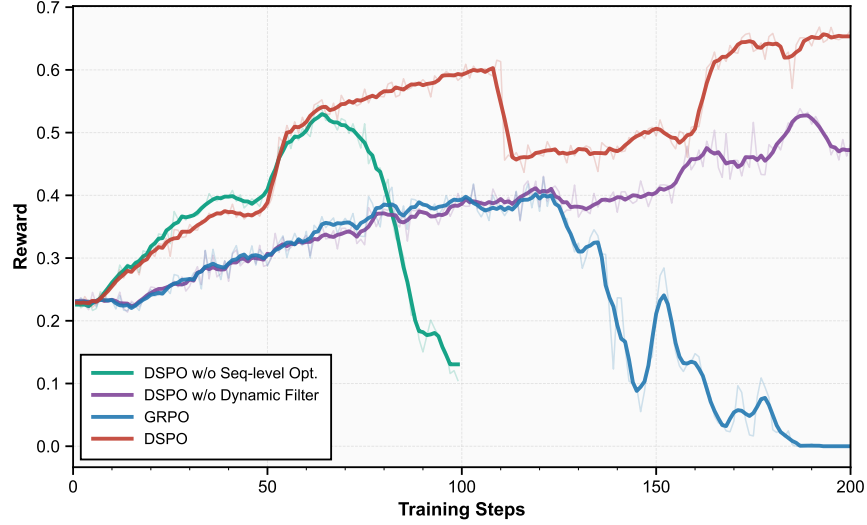


Figure 3: **Training reward dynamics of DSPO and its ablations.** DSPO (red) achieves stable, monotonic improvement. In contrast, token-level variants (green, blue) collapse catastrophically, while the sequence-level variant without dynamic filtering (purple) remains stable but plateaus at a suboptimal level.

Prompt Template. Answer the given question. You must conduct reasoning inside `<think>` and `</think>`. first every time you get new information. After reasoning, if you find you lack some knowledge, you can call a search engine by `<tool_call>` query `</tool_call>` and it will return the top searched results between `<tool_response>` and `</tool_response>`. You can search as many times as your want. If you find no further external knowledge needed, you can directly provide the answer inside `<answer>` and `</answer>`, without detailed illustrations. For example, `<answer>` Beijing `</answer>`. **Question:** ...

Table 1: Prompt template used to structure reasoning, search calls, and final answers in our experiments.

For implementation, we use Qwen2.5-7B-Instruct as the starting checkpoint for all training. Our experiments build on the VeRL framework (Sheng et al., 2025), adapting its Search-R1-like example code and scripts to implement our methodology. Because DSPO uses a modified reward function, we retrain the PPO and GRPO Search-R1 baselines (Jin et al., 2025b;a) under the same experimental conditions. These retrained models provide the primary external benchmarks and ensure that comparisons reflect optimization quality rather than reward or implementation differences.

Implementation and Evaluation. To isolate the algorithmic contribution, all RL experiments deliberately use a standard BM25 retriever. This controlled setup attributes observed improvements to the learned policy rather than a stronger retrieval backend. Across all methods, models are trained with a sparse binary reward based on substring Exact Match (subEM) of the final answer, and subEM is the primary evaluation metric.

4.2 MAIN RESULTS AND ABLATION STUDY

Table 2 gives a comprehensive view of DSPO’s effectiveness. Because DSPO’s gains come from the interaction of its two components, we present main results and ablations together. This single comparison places DSPO against strong external baselines and its own ablated variants, directly testing both overall performance and the necessity of each component.

Comparison with Baselines. Table 2 demonstrates DSPO’s clear advantage. The DSPO-trained 7B agent achieves an average score of 0.531, a 34.1% relative improvement over the same-sized Search-R1 (GRPO, 7B) model. More importantly, the 7B DSPO agent slightly exceeds the average score of the much larger Search-R1 14B models trained with either GRPO or PPO. This result strongly indicates that the gains come from a more effective and stable learning algorithm rather than from increased model scale.

Table 2: Comprehensive comparison of DSPO, external baselines, and ablation variants on seven QA benchmarks. Baselines include Search-R1 models (7B & 14B) trained with GRPO and PPO (Jin et al., 2025b;a). Ablations remove DSPO’s key components: ‘w/o dynamic filter’ and ‘w/o seq-level opt.’. Original EM scores from Search-R1 are in parentheses. For evaluation consistency, we retrained and evaluated the baselines with our adjusted rewards. Best results are in **bold**; second-best are underlined.

Dataset	Search-R1				DSPO & Ablations (Ours, 7B)		
	GRPO (7B)	PPO (7B)	GRPO (14B)	PPO (14B)	w/o dyn. filter	w/o seq-lvl opt.	DSPO
NQ	0.423 (0.429)	(0.393)	<u>0.535</u> (0.482)	(0.424)	0.363	0.470	0.580
TriviaQA	0.658 (0.623)	(0.610)	0.760 (0.667)	(0.660)	0.515	0.695	<u>0.754</u>
PopQA	0.395 (0.427)	(0.397)	<u>0.477</u> (0.434)	(0.442)	0.277	0.430	0.498
HotpotQA	0.401 (0.386)	(0.370)	<u>0.563</u> (0.429)	(0.436)	0.330	0.438	0.613
2WikiMultiHopQA	0.357 (0.414)	(0.346)	0.611 (0.424)	(0.379)	0.285	0.398	<u>0.569</u>
Musique	0.122 (0.162)	(0.146)	<u>0.260</u> (0.191)	(0.210)	0.105	0.133	0.270
Bamboogle	0.280 (0.400)	(0.368)	0.504 (0.492)	(0.480)	0.288	0.280	<u>0.432</u>
Average	0.377 (0.396)	(0.385)	<u>0.530</u> (0.446)	(0.433)	0.313	0.406	0.531

Analysis of Ablations. The ablation results in Table 2 show that both DSPO components are necessary. Removing the dynamic filter (‘w/o dynamic filter’) causes a severe performance drop, reducing the average score to 0.313. This confirms the filter’s critical role: without it, the sequence-level objective is starved of useful learning signal by homogeneous-reward batches. Ablating sequence-level optimization (‘w/o sequence-level opt.’) also substantially degrades performance, yielding an average score of 0.406. Although this token-level variant outperforms the filter-less model, it remains far below full DSPO and, as the next section shows, is vulnerable to catastrophic training instability. These results establish the synergy: sequence-level updates provide stability, while dynamic filtering converts sparse rewards into efficient learning signal.

Beyond aggregate metrics, DSPO produces sophisticated search behaviors, including recognition of irrelevant results, query reformulation, and multi-turn verification (see Appendix A for detailed trajectory examples). These behaviors emerge from pure RL training with DSPO rather than from supervised demonstrations.

4.3 ANALYSIS OF TRAINING DYNAMICS

To validate DSPO’s stability and efficiency claims, we analyze training reward dynamics for DSPO, its ablations, and key baselines. Figure 3 makes the contrast clear and reinforces the core architectural choices behind DSPO.

DSPO (red) follows a smooth, monotonic ascent and converges efficiently to the highest reward level. This trajectory empirically confirms the stability delivered by the sequence-level objective. In sharp contrast, the **token-level methods**—DSPO w/o Seq-level Opt. (green) and vanilla GRPO (blue)—undergo catastrophic policy collapse early in training. Their rewards fall after a short initial improvement, directly reflecting the instability induced by high-variance token-level gradient updates. Meanwhile, **DSPO w/o Dynamic Filter** (purple) remains stable because it uses sequence-level updates, but it plateaus at a clearly suboptimal ceiling because it lacks an efficient learning signal. These dynamics demonstrate that DSPO’s combination of sequence-level stability and dynamic filtering is the key driver of robust and effective policy optimization.

To verify that reward improvements reflect genuine generalization rather than reward hacking, we track validation performance on key benchmarks throughout training. As shown in Figure 2, DSPO’s validation accuracy on NQ, HotpotQA, and other diverse benchmarks rises consistently with its stable reward curve. This alignment confirms that the agent learns a generalizable search-and-reasoning policy.

4.4 SCALABILITY AND GENERALIZATION ANALYSIS

We further test DSPO’s robustness by evaluating both model-scale transfer and domain generalization.

Scalability to Larger Models. We test whether DSPO’s stability benefits persist at larger parameter scales by training Qwen2.5-14B-Instruct. Table 3 shows that DSPO scales effectively: the DSPO-trained 14B model reaches 60.6%

average accuracy and significantly outperforms the strong GRPO-14B baseline (53.0%) by a **14.3%** relative margin. These results confirm that DSPO leverages increased model capacity while consistently exceeding standard baselines.

Table 3: Scalability analysis showing DSPO’s gains on Qwen2.5-14B-Instruct. Best results are in **bold**.

Dataset	Instruct (14B)	GRPO (14B)	DSPO (7B)	DSPO (14B)	Gain
NQ	0.345	0.535	0.580	0.629	+17.6%
HotpotQA	0.407	0.563	0.613	0.665	+18.1%
2WikiMQA	0.332	0.611	0.569	0.699	+14.4%
Bamboogle	0.328	0.504	0.432	0.544	+7.9%
PopQA	0.364	0.477	0.498	0.545	+14.3%
TriviaQA	0.643	0.760	0.754	0.802	+5.5%
Musique	0.151	0.260	0.270	0.361	+38.8%
Average	0.367	0.530	0.531	0.606	+14.3%

Generalization to Mathematical Reasoning. We further test the generality of DSPO by applying it to single-turn mathematical reasoning tasks with the Qwen2.5 and Qwen3 model families. Table 4 reports results on Math500 and Olympiad-Bench. DSPO consistently surpasses GRPO at both 7B and 4B scales, indicating that its optimization principles extend beyond agentic search to broader reasoning domains.

Table 4: Generalization results for mathematical reasoning, where DSPO consistently improves over GRPO. Best results are in **bold**.

Model	Benchmark	Steps	GRPO	DSPO	Gain
Qwen2.5-Math-7B	Math500	200	0.772	0.798	+2.6%
Qwen3-4B	Olympiad-Bench	100	0.728	0.755	+2.7%

5 CONCLUSION

This work addresses the critical instability and sample-inefficiency problems that limit RL training for autonomous LLM search agents. We introduced Dynamic-filter Sequence-level Policy Optimization (DSPO), a robust algorithm built on two decisive components: sequence-level optimization that prevents catastrophic policy collapse, and dynamic outcome-based filtering that converts sparse rewards into a consistently effective learning signal. Across challenging question-answering benchmarks, DSPO achieves strong performance while exhibiting markedly superior training stability compared with prior methods.

By enabling robust training from environmental feedback alone, DSPO provides a practical and efficient blueprint for building capable LLM agents without costly expert demonstrations. This stable foundation opens a clear path to integrating stronger retrievers and extending DSPO to complex multi-tool tasks. Because sparse rewards and unstable policy gradients also arise beyond search, DSPO’s principles are well positioned to benefit domains such as mathematics and code generation, a direction that warrants further validation. The central lessons of DSPO—matching the optimization unit to the reward signal and guaranteeing signal density—provide a strong basis for developing the next generation of autonomous AI.

REFERENCES

- Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020.
- Tuhin Chakrabarty, Vishakh Padmakumar, Faeze Brahman, and Smaranda Muresan. Creativity support in the age of large language models: An empirical study involving professional writers. In *Proceedings of the 16th Conference on Creativity & Cognition*, pp. 132–155, 2024.

- Mingyang Chen, Tianpeng Li, Haoze Sun, Yijie Zhou, Chenzheng Zhu, Haofen Wang, Jeff Z Pan, Wen Zhang, Huajun Chen, Fan Yang, et al. Learning to reason with search for llms via reinforcement learning. *arXiv preprint arXiv:2503.19470*, 2025.
- Paul F Christiano, Jan Leike, Tom Brown, Miljan Martic, Shane Legg, and Dario Amodei. Deep reinforcement learning from human preferences. *Advances in neural information processing systems*, 30, 2017.
- Tianzhe Chu, Yuexiang Zhai, Jihan Yang, Shengbang Tong, Saining Xie, Dale Schuurmans, Quoc V Le, Sergey Levine, and Yi Ma. Sft memorizes, rl generalizes: A comparative study of foundation model post-training. *arXiv preprint arXiv:2501.17161*, 2025.
- Ganqu Cui, Yuchen Zhang, Jiacheng Chen, Lifan Yuan, Zhi Wang, Yuxin Zuo, Haozhan Li, Yuchen Fan, Huayu Chen, Weize Chen, et al. The entropy mechanism of reinforcement learning for reasoning language models. *arXiv preprint arXiv:2505.22617*, 2025.
- Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yixin Dai, Jiawei Sun, Haofen Wang, and Haofen Wang. Retrieval-augmented generation for large language models: A survey. *arXiv preprint arXiv:2312.10997*, 2(1), 2023.
- Xanh Ho, Anh-Khoa Duong Nguyen, Saku Sugawara, and Akiko Aizawa. Constructing a multi-hop qa dataset for comprehensive evaluation of reasoning steps. *arXiv preprint arXiv:2011.01060*, 2020.
- Bowen Jin, Jinsung Yoon, Priyanka Kargupta, Sercan O Arik, and Jiawei Han. An empirical study on reinforcement learning for reasoning-search interleaved llm agents. *arXiv preprint arXiv:2505.15117*, 2025a.
- Bowen Jin, Hansi Zeng, Zhenrui Yue, Jinsung Yoon, Sercan Arik, Dong Wang, Hamed Zamani, and Jiawei Han. Search-rl: Training llms to reason and leverage search engines with reinforcement learning. *arXiv preprint arXiv:2503.09516*, 2025b.
- Mandar Joshi, Eunsol Choi, Daniel S Weld, and Luke Zettlemoyer. Triviaqa: A large scale distantly supervised challenge dataset for reading comprehension. *arXiv preprint arXiv:1705.03551*, 2017.
- Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick SH Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. Dense passage retrieval for open-domain question answering. In *EMNLP (1)*, pp. 6769–6781, 2020.
- Tom Kwiatkowski, Jennimaria Palomaki, Olivia Redfield, Michael Collins, Ankur Parikh, Chris Alberti, Danielle Epstein, Illia Polosukhin, Jacob Devlin, Kenton Lee, et al. Natural questions: a benchmark for question answering research. *Transactions of the Association for Computational Linguistics*, 7:453–466, 2019.
- Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in neural information processing systems*, 33:9459–9474, 2020.
- Zichen Liu, Changyu Chen, Wenjun Li, Penghui Qi, Tianyu Pang, Chao Du, Wee Sun Lee, and Min Lin. Understanding rl-zero-like training: A critical perspective. *arXiv preprint arXiv:2503.20783*, 2025.
- Alex Mallen, Akari Asai, Victor Zhong, Rajarshi Das, Hannaneh Hajishirzi, and Daniel Khashabi. When not to trust language models: Investigating effectiveness and limitations of parametric and non-parametric memories. *arXiv preprint arXiv:2212.10511*, 7, 2022.
- Guillermo Marco, Julio Gonzalo, Ramón del Castillo, and María Teresa Mateo Girona. Pron vs prompt: Can large language models already challenge a world-class fiction author at creative text writing? *arXiv preprint arXiv:2407.01119*, 2024.
- Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. *Advances in neural information processing systems*, 35:27730–27744, 2022.
- Ofir Press, Muru Zhang, Sewon Min, Ludwig Schmidt, Noah A Smith, and Mike Lewis. Measuring and narrowing the compositionality gap in language models. *arXiv preprint arXiv:2210.03350*, 2022.
- Rafael Rafailov, Archit Sharma, Eric Mitchell, Christopher D Manning, Stefano Ermon, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. *Advances in neural information processing systems*, 36:53728–53741, 2023.

- Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. *Advances in Neural Information Processing Systems*, 36:68539–68551, 2023.
- John Schulman, Philipp Moritz, Sergey Levine, Michael Jordan, and Pieter Abbeel. High-dimensional continuous control using generalized advantage estimation. *arXiv preprint arXiv:1506.02438*, 2015.
- John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, YK Li, Yang Wu, et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.
- Guangming Sheng, Chi Zhang, Zilingfeng Ye, Xibin Wu, Wang Zhang, Ru Zhang, Yanghua Peng, Haibin Lin, and Chuan Wu. Hybridflow: A flexible and efficient rlhf framework. In *Proceedings of the Twentieth European Conference on Computer Systems*, pp. 1279–1297, 2025.
- Huatong Song, Jinhao Jiang, Yingqian Min, Jie Chen, Zhipeng Chen, Wayne Xin Zhao, Lei Fang, and Ji-Rong Wen. R1-searcher: Incentivizing the search capability in llms via reinforcement learning. *arXiv preprint arXiv:2503.05592*, 2025.
- Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023.
- Trieu H Trinh, Yuhuai Wu, Quoc V Le, He He, and Thang Luong. Solving olympiad geometry without human demonstrations. *Nature*, 625(7995):476–482, 2024.
- Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. Musique: Multihop questions via single-hop question composition. *Transactions of the Association for Computational Linguistics*, 10:539–554, 2022.
- Junde Wu, Jiayuan Zhu, Yuyuan Liu, Min Xu, and Yueming Jin. Agentic reasoning: A streamlined framework for enhancing llm reasoning with agentic tools. *arXiv preprint arXiv:2502.04644*, 2025.
- Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, et al. Autogen: Enabling next-gen llm applications via multi-agent conversations. In *First Conference on Language Modeling*, 2024.
- John Yang, Carlos E Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. Swe-agent: Agent-computer interfaces enable automated software engineering. *Advances in Neural Information Processing Systems*, 37:50528–50652, 2024.
- Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W Cohen, Ruslan Salakhutdinov, and Christopher D Manning. Hotpotqa: A dataset for diverse, explainable multi-hop question answering. *arXiv preprint arXiv:1809.09600*, 2018.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.
- Qiyang Yu, Zheng Zhang, Ruofei Zhu, Yufeng Yuan, Xiaochen Zuo, Yu Yue, Weinan Dai, Tiantian Fan, Gaohong Liu, Lingjun Liu, et al. Dapo: An open-source llm reinforcement learning system at scale. *arXiv preprint arXiv:2503.14476*, 2025.
- Wayne Xin Zhao, Kun Zhou, Junyi Li, Tianyi Tang, Xiaolei Wang, Yupeng Hou, Yingqian Min, Beichen Zhang, Junjie Zhang, Zican Dong, et al. A survey of large language models. *arXiv preprint arXiv:2303.18223*, 1(2), 2023.
- Yuzhong Zhao, Yue Liu, Junpeng Liu, Jingye Chen, Xun Wu, Yaru Hao, Tengchao Lv, Shaohan Huang, Lei Cui, Qixiang Ye, et al. Geometric-mean policy optimization. *arXiv preprint arXiv:2507.20673*, 2025.
- Chujie Zheng, Shixuan Liu, Mingze Li, Xiong-Hui Chen, Bowen Yu, Chang Gao, Kai Dang, Yuqiong Liu, Rui Men, An Yang, et al. Group sequence policy optimization. *arXiv preprint arXiv:2507.18071*, 2025.
- Qinkai Zheng, Xiao Xia, Xu Zou, Yuxiao Dong, Shan Wang, Yufei Xue, Lei Shen, Zihan Wang, Andi Wang, Yang Li, et al. Codegeex: A pre-trained model for code generation with multilingual benchmarking on humaneval-x. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, pp. 5673–5684, 2023.

A APPENDIX

Table A.1 gives representative trajectories that make DSPO’s learned search strategies concrete. The examples show that the model can: (1) reformulate queries when initial searches return irrelevant results, (2) verify information through follow-up searches, and (3) extract comprehensive answers from search results.

Table A.1: Representative trajectories demonstrating the model’s learned search and reasoning capabilities. DSPO produces sophisticated behaviors including (a) recognition of irrelevant results, (b) query reformulation, and (c) verification strategies.

Example 1: Presidential Succession (<i>Query Refinement after Failure</i>)	
Question	Who is next in line if something happens to the president?
Gold	vice president
Turn 1	<code>search("who is next in line if something happens to the president")</code> → Retrieved: Documents about novels "Something Happened" and "Mosquitoes Buzz" Agent: "The queries were not directly about the line of succession... The relevant information I need is not present in the search results. Let’s try searching again with a more specific query."
Turn 2	<code>search("line of succession for the president")</code> → Retrieved: Philippine and US presidential succession information Agent: "Based on the search results... the next person in line would be the Vice President."
Answer	✓ The Vice President
Example 2: University Identification (<i>Information Verification</i>)	
Question	Which American University is situated in Ithaca, New York?
Gold	Cornell University
Turn 1	<code>search(["American University in Ithaca", "University in Ithaca NY"])</code> → Retrieved: "Ithaca is home to Cornell University, an Ivy League school..." Agent: "Ithaca, New York is home to Cornell University, which is an Ivy League school. However... it is highly likely that the university in question is Cornell. Let me confirm this."
Turn 2	<code>search(["Cornell University location", "Cornell University New York"])</code> → Retrieved: Cornell history confirming establishment in Ithaca Agent: "Cornell University is indeed located in Ithaca, New York."
Answer	✓ Cornell University
Example 3: Biographical Information (<i>Direct Extraction</i>)	
Question	What is John Andrew Martin’s occupation?
Gold	lawyer, attorney, politician
Turn 1	<code>search(["John Andrew Martin occupation", "John Andrew Martin job"])</code> → Retrieved: "American journalist, attorney, soldier, and politician..." Agent: "John Andrew Martin was an American <u>journalist, attorney, soldier, and politician</u>. He represented Colorado in the U.S. House of Representatives."
Answer	✓ Journalist, attorney, soldier, and politician